



Banking AI Agent

Contextual Engineering in Practice

LangChain · LangGraph · OpenAI · RAG · Multi-Agent

Demonstrating all four Contextual Engineering strategies through a production-style banking assistant

The Core Problem with LLM Agents

LLMs have a **fixed context window** — they can only "see" what you put in front of them.

As conversations grow, agents face three critical failures:

Failure	What Happens	Impact
 No Memory	Each turn starts fresh	User repeats account ID every time
 Context Overflow	Too many tokens in one prompt	API errors, high cost
 Context Pollution	Irrelevant info fills the window	Poor, hallucinated answers

Solution: Contextual Engineering — deliberately manage *what* the LLM sees, *when*, and *how much*.

What is Contextual Engineering?

"The discipline of designing and managing the information presented to an LLM to maximise the quality of its outputs."

Four core strategies:

-  WRITE → Persist important facts to memory
-  SELECT → Retrieve only what's relevant now
-  COMPRESS → Summarise to fit within context limit
-  ISOLATE → Separate agents with scoped contexts

This project implements **all four** in a banking customer experience agent.

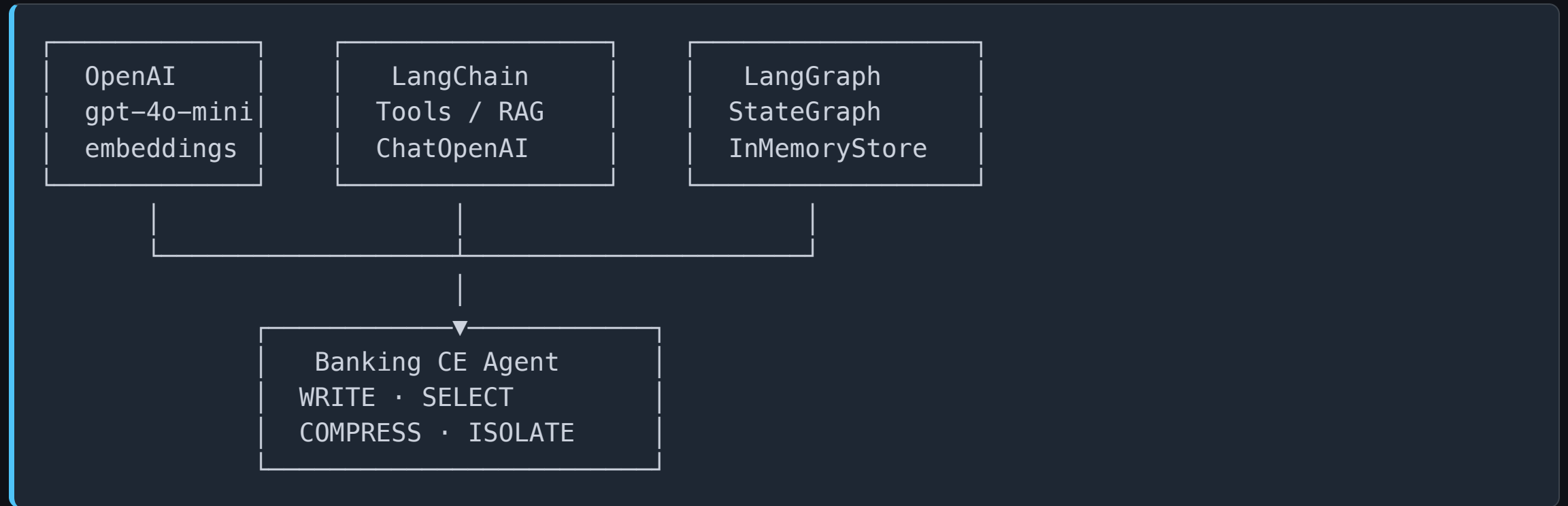
The Banking CE Agent

A fully functional banking assistant for **IndiaFirst Bank** that handles:

- 💰 Account balance & transaction history
- 🚨 Fraud detection & suspicious transaction flagging
- 🏠 Loan eligibility checks (personal & home loans)
- 📊 EMI calculations & Fixed Deposit rates
- 📄 UPI limit checks & transfer validation
- 🎫 Support ticket creation
- 📖 Policy Q&A via RAG (regulatory, KYC, limits)

Three customers with realistic profiles: Rajesh Kumar, Priya Sharma, Amit Patel

Technology Stack



Library	Role
<code>langchain-openai</code>	LLM & embeddings
<code>langgraph</code>	Agent graph, state, memory store
<code>faiss-cpu</code>	In-memory vector search for RAG

Strategy 1: WRITE

Persist important facts so the agent remembers them across turns and sessions.

Two layers of memory:

Layer 1 — Session Scratchpad (StateGraph)

```
class BankingState(TypedDict):
    messages: list      # full conversation
    current_account: str # ← active account, persisted across turns
    fraud_flags: list    # ← flagged transactions this session
    loan_context: dict   # ← loan evaluation in progress
    session_summary: str  # rolling summary
    interaction_count: int # turn counter
```

Layer 2 — Long-Term Cross-Session Store (InMemoryStore)

```
store.put(("banking_sessions", account_id), "context", {
    "summary": new_summary,
    "interactions": interactions[-5:]
})
```

Strategy 2: SELECT

Retrieve only the context that is relevant for the current query — don't dump everything in.

Two SELECT mechanisms:

1. RAG — Policy Document Retrieval

```
vectorstore = FAISS.from_documents(policy_docs, OpenAIEmbeddings())  
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

The agent calls `search_banking_policy` only when it needs loan rules, UPI limits, KYC requirements, or fraud detection thresholds.

2. Memory Injection — Prior Session Context

```
# Only injected if a prior summary exists for this account  
if prior_summary:  
    system_prompt += f"\n[Memory for {account_id}]\n{prior_summary}"
```

The agent never sees *all* historical data — only what's *relevant now*.

Strategy 3: COMPRESS

Summarise verbose history into a compact representation that fits within the context window.

Triggered every 3 interactions per account:

```
def compress_node(state, store):
    summary = llm.invoke([
        SystemMessage(content=COMPRESS_PROMPT),
        *state["messages"]
    ])

    # ✅ Save compact summary to long-term store
    store.put(("banking_sessions", account_id), "summary", summary)

    # ✅ Drop old messages – keep only last 6
    return {
        "session_summary": summary,
        "messages": state["messages"][-6:]
    }
```

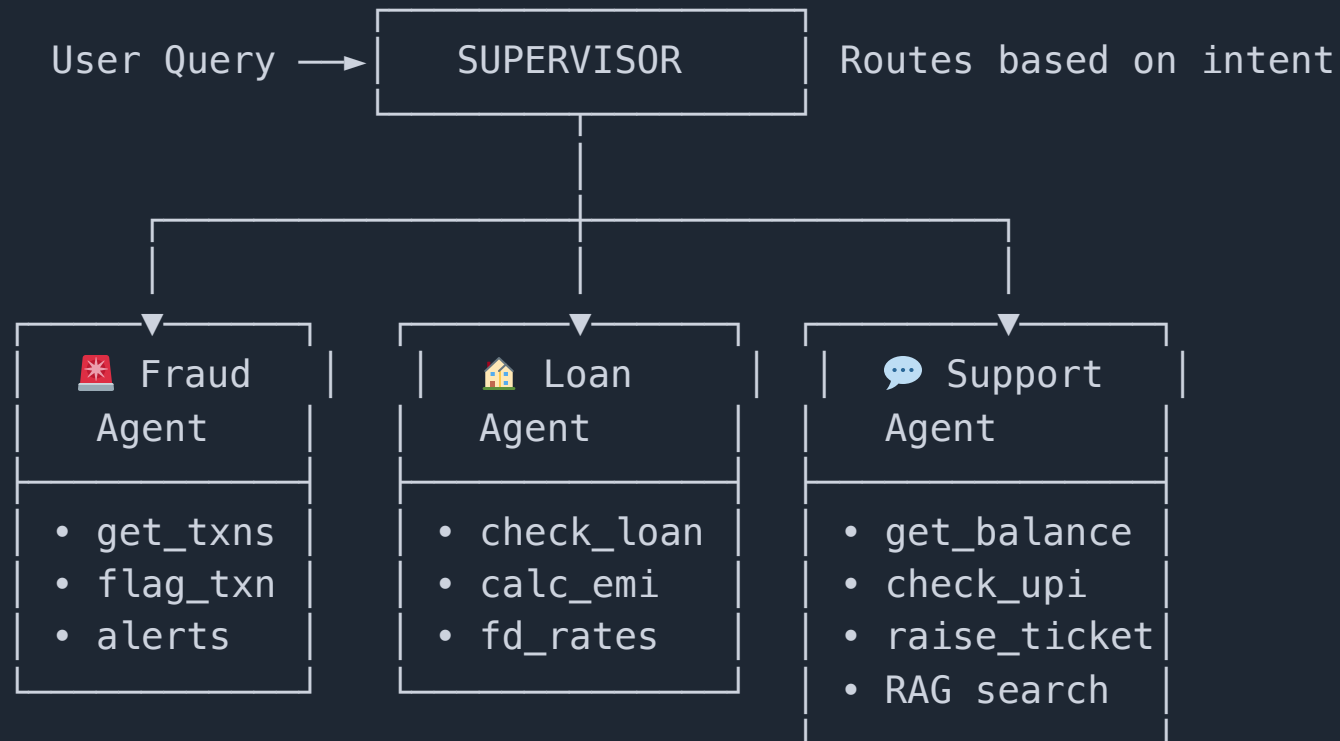
Before COMPRESS: 30+ messages in context

After COMPRESS: 1 paragraph summary + last 6 messages



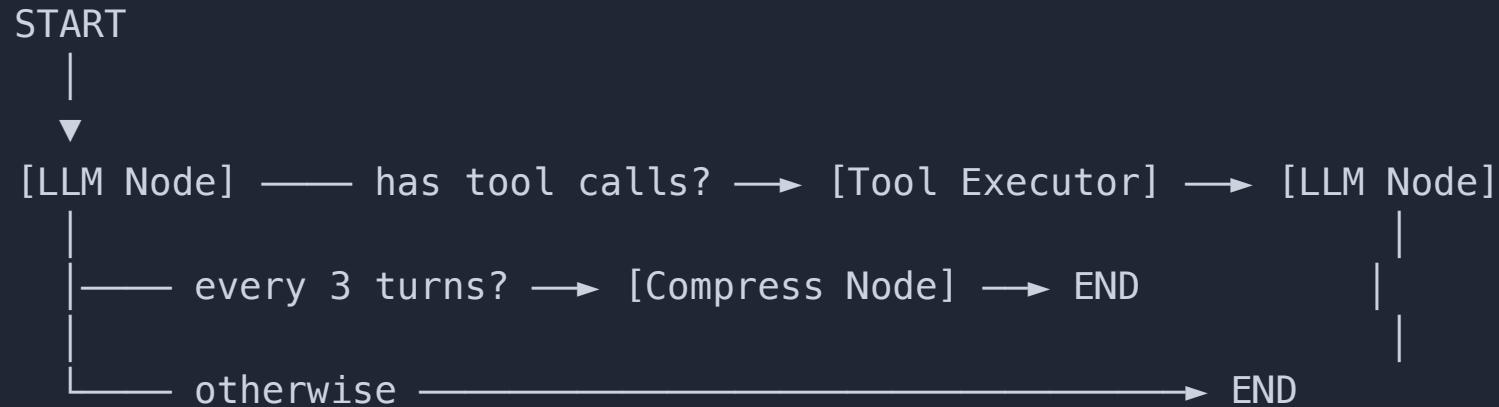
Strategy 4: ISOLATE

Give each specialist agent only the tools and context it needs — nothing more.



Each agent has its own isolated context — Fraud Agent never sees loan data, Loan Agent never sees support history.

Single Agent — Graph Architecture



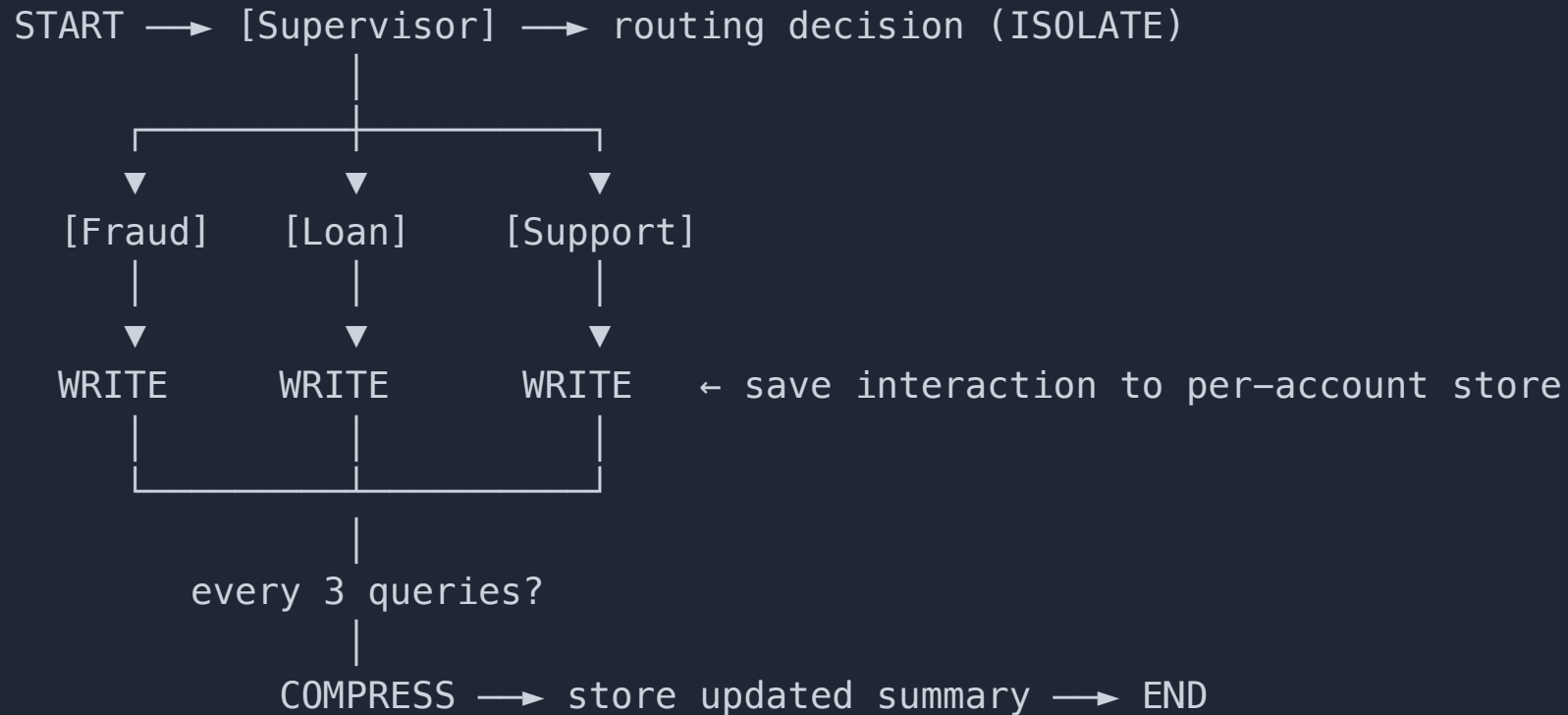
LLM Node does:

- Injects session scratchpad + prior memory (WRITE + SELECT)
- Calls LLM with all tools bound including RAG retriever (SELECT)
- Extracts and persists account ID from messages (WRITE)

Tool Executor does:

- Runs each tool call, returns ToolMessage with `tool_call_id`
- Updates `fraud_flags` and `loan_context` scratchpad (WRITE)

Supervisor — Multi-Agent Architecture



Account isolation: namespace `("supervisor_sessions", "ACC001")` is completely separate from `("supervisor_sessions", "ACC002")`.



Demo Flow — Single Agent

Turn	User	Concepts Active
1	"My account is ACC001. Check balance."	WRITE (stores ACC001)
2	"Suspicious Rs.1,20,000 transfer on Mar 12."	SELECT (loads ACC001 context), WRITE (flags fraud)
3	"Am I eligible for a Rs.10L personal loan?"	COMPRESS fires (turn 3), SELECT (RAG loan policy)
4	"EMI at 10.5% for 48 months?"	WRITE (loan context persisted)
5	"What are UPI limits for Rs.90,000 transfer?"	SELECT (RAG UPI policy)

Watch for in terminal:

- 🔍 **Cyan panels** = SELECT firing
- 🖋️ **Yellow panels** = WRITE saving
- 🏗️ **Green panels** = COMPRESS triggered
- 🔄 **Magenta panels** = ISOLATE routing



Demo Flow — Supervisor Agent

Query	Account	Routed To	Concepts
"Rs.1,20,000 at 2AM — fraud?"	ACC001	 Fraud Agent	ISOLATE routes, WRITE saves
"Eligible for home loan of Rs.50L?"	ACC001	 Loan Agent	ISOLATE routes, SELECT injects ACC001 memory
"Balance + fraud alerts?"	ACC002	 Support Agent	ISOLATE routes, WRITE saves (separate namespace)

Key observation:

- ACC001 and ACC002 memories are **never mixed** — separate `InMemoryStore` namespaces
- Sub-agents receive HumanMessages **plus a memory** `SystemMessage` if prior context exists (SELECT within isolation)
- Supervisor routing decisions are hidden from sub-agents — true context isolation
- After 3 queries **per account** → **COMPRESS** panel appears in green; interaction log resets

Code: WRITE Strategy

```
# — In banking_llm_node —————  
import re  
current_account = state.get("current_account")  
  
if not current_account:  
    for msg in state["messages"]:  
        match = re.search(r"ACC\d{3,}", str(msg.content))  
        if match:  
            current_account = match.group(0) # ← auto-extract account ID  
            break  
  
if current_account:  
    updates["current_account"] = current_account # ← persist to scratchpad  
  
# — Cross-session long-term memory —————  
store.put(  
    ("banking_sessions", account_id), # ← scoped by account_id  
    "summary",  
    {"summary": new_summary}  
)
```

Code: SELECT & COMPRESS

```
# — SELECT: RAG retrieval (only called when agent needs policy) —
retriever_tool = Tool(
    name="search_banking_policy",
    func=lambda q: "\n\n".join(
        doc.page_content
        for doc in retriever.invoke(q)
    )
)

# — SELECT: Memory injection —
memories = store.search("banking_sessions", account_id)
if memories:
    system_prompt += f"\n[Prior session]\n{memories[0].value['summary']}"

# — COMPRESS: Triggered every 3 turns —
if interaction_count % 3 == 0:
    summary = llm.invoke([SystemMessage(COMPRESS_PROMPT)] + messages)
    store.put(namespace, "summary", {"summary": summary.content})
    trimmed = messages[-6:]    # ← drop old messages after summarising
```

Code: ISOLATE Strategy

```
# Each agent gets ONLY its domain tools – no cross-contamination
fraud_agent = create_react_agent(
    model=llm,
    tools=[get_transaction_history,      # ← fraud domain only
           flag_suspicious_transaction,
           get_active_alerts],
    prompt=FRAUD_AGENT_PROMPT,
)

loan_agent = create_react_agent(
    model=llm,
    tools=[check_loan_eligibility,      # ← loan domain only
           calculate_emi, get_fd_rates, get_customer_profile],
)

# Sub-agents get HumanMessages + injected memory (SELECT within isolation)
# Supervisor routing tool calls are never passed through
def _human_messages_with_context(state):
    human_msgs = [m for m in state["messages"] if isinstance(m, HumanMessage)]
    if prior_summary: # ← SELECT: inject stored memory per account
        return [SystemMessage(content=prior_summary)] + human_msgs
    return human_msgs
```


What Makes This Production-Quality

Feature	Implementation
Per-user memory isolation	Namespaced <code>InMemoryStore</code> keyed by <code>account_id</code>
No repeated account ID asks	Auto-extracted from messages via regex, stored in state
Valid tool call history	Every <code>tool_call_id</code> matched with a <code>ToolMessage</code>
RAG over policies	FAISS vector store, top-3 semantic retrieval
Context window management	Trim to last 6 messages after compression
Graceful fallback	Missing account ID → prompt only at tool call time
Multi-model support	Swap model string to use Claude, Gemini, etc.



Memory is Context, Not a Cache

Common misconception: "Storing in memory means the LLM is skipped for repeated questions."

Reality: Memory here is *injected into the prompt* — the LLM always runs.

Why? Deliberate design choice for banking:

- ✗ Return cached "balance: ₹1,23,456" → stale, potentially harmful
- ✓ Call `get_account_balance()` live → fresh, accurate, safe

The terminal makes this explicit on every query:

- This context is INJECTED into the LLM prompt.
- Tools will still be called for fresh, live data — memory does NOT replace tool calls.

What memory *does* provide:

- Customer doesn't repeat account ID, fraud history, loan status every session
- Cross-session continuity — returning customer picks up where they left off
- Per-account namespace isolation — ACC001 context never leaks to ACC002



How COMPRESS Saves LLM Cost

COMPRESS is the real cost optimisation — it keeps token count **bounded and predictable**.

Without COMPRESS: tokens grow without limit

```
Turn 1: ~400 tokens
Turn 5: ~2,500 tokens
Turn 10: ~8,000 tokens ← cost spikes
Turn 30: ~25,000 tokens ← approaching context limits
```

With COMPRESS: tokens stay flat after every 3 turns

```
Turn 1: ~400 tokens
Turn 3: compress → 1 summary + last 6 messages → ~1,500 tokens
Turn 6: compress → 1 summary + last 6 messages → ~1,500 tokens
Turn 30: compress → 1 summary + last 6 messages → ~1,500 tokens ✓
```

What compress_node does:

1. Sends full history to LLM → 3–5 sentence summary (COMPRESS)
2. Persists summary to `InMemoryStore` (WRITE)

Project Structure

```
agent-demo/  
├── banking_ce_agent/  
│   ├── banking_agent.py      ← Main agent: graph, nodes, strategies  
│   ├── banking_tools.py      ← 10 banking tools (@tool decorated)  
│   └── banking_knowledge.py   ← Policy docs for RAG (loans, fraud, UPI...)  
├── requirements.txt          ← LangChain, LangGraph, FAISS, rich  
├── .env                      ← OPENAI_API_KEY  
└── readme.md
```

Run the demos:

```
# Interactive chat – supervisor mode (default)  
python banking_ce_agent/banking_agent.py  
python banking_ce_agent/banking_agent.py chat  
  
# Interactive chat – single-agent mode  
python banking_ce_agent/banking_agent.py chat single  
  
# Scripted single-agent demo (WRITE · SELECT · COMPRESS)  
python banking_ce_agent/banking_agent.py single
```

Key Takeaways

Contextual Engineering is not a framework — it's a discipline.

Without CE	With CE
Agent asks for account ID every turn	Account ID captured once, persisted for session
Loan policies hallucinated	Exact policy retrieved via RAG
Context grows → API errors	Rolling compression keeps window bounded
One agent knows too much	Each specialist agent isolated to its domain
User A sees User B's data	Per-user namespaced memory store

The quality of your agent's output is directly proportional to the quality of context you provide it.

Thank You

Banking CE Agent — Full Implementation

GitHub: github.com/RaviChandraSingam/agent-demo

Four strategies, one demo:

- ✍️ WRITE → LangGraph StateGraph + InMemoryStore
- 🔍 SELECT → FAISS RAG + Memory injection
- 📄 COMPRESS → Rolling summary + message trimming
- 🔗 ISOLATE → Supervisor + domain-scoped specialist agents

Run it yourself:

```
git clone https://github.com/RaviChandraSingam/agent-demo
cd agent-demo && python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
echo "OPENAI_API_KEY=sk-..." > .env

python banking_ce_agent/banking_agent.py          # interactive chat (default)
python banking_ce_agent/banking_agent.py both      # run both scripted demos
```